05/11/2025

Competitive Programmer's Handbook

Antti Laaksonen

Chapter 1:

Introduction:

Competitive programming → Design of algorithms. (1)
→ Implemenation of algorithms. (2)

(1) Consists of problem solving and mathematical thinking.

An algorithm has to be both correct and efficient, and the core of the problem is often to create an efficient algorithm.

→ Theorical knowledge of algorithms is important,

Typically, a solution to a problem is a combination of well-known techniques.

(2) Requires good programming skills, programs should be written quickly, because there is not much time available. (A few hundred lines of code max).

Programming Languages:

The most popular programming languages used are: C++ (79%), Python (16%) and Java (8%).

Many people thinks that C++ is the best choice, because it's very efficient and its standard library contains a large collection of data structures and algorithms.

The programs follow the C++11 standard.

C++ code template:

```
#include <bits/stdc++.h>
using namespace std;

int main(){
    // solution comes here
}
```

→ Include the entire standard library. (No need to separately include libraries such as iostream, vector and algorithm)

→ Without this line we would have to write std::cout, but now it suffices to write only cout.

INPUT AND OUTPUT:

IN C++, THE STANDARD STREAMS ARE CIN FOR INPUT E COUT FOR OUTPUT. (SCANF AND PRINTF CAN BE USED).

FOR EXAMPLE:

```
int a, b;
string x;
cin >> a >> b >> x;
```

→ TAKES INPUT WITH AT LEAST ONE SPACE OR NEWLINE BETWEEN EACH ELEMENT

→ 123 (1 SPACE) 456 (1 SPACE) MONKEY

OR

→ 123 (3 SPACES) 456 (1 NEWLINE) MONKEY

THE COUT IS USED FOR OUTPUT AS FOLLOWS:

```
int a = 123, b = 456;
string x = "monkey";
cout << a << " " << b << " " << x << "\n";
```

→ PRINTS: 123 456 MONKEY

NOTE: INPUT AND OUTPUT IS SOMETIMES A BOTTLENECK IN THE PROGRAM, THE FOLLOWING LINES AT THE BEGINNING OF THE CODE MAKE THEM MORE EFFICIENT:

```
ios::sync_with_stdio(0);
cin.tie(0);
```

NOTE: THE "\N" WORKS FASTER THAN ENDL, BECAUSE ENDL ALWAYS CAUSES A FLUSH OPERATION.

THE C FUNCTIONS SCANF AND PRINTF CAN BE USE TOO, THEY'RE USALLY A BIT FASTER:

```
int a, b;
scanf("%d %d", &a, &b);
```

→ CIN IN C++

```
int a = 123, b = 456;
printf("%d %d\n", &a, &b);
```

→ COUT IN C++

SOMETIMES THE PROGRAM SHOULD READ A WHOLE LINE FROM THE INPUT (POSSIBLY CONTAINING SPACES):

```
string s;
getline(cin, s);
```

→ TAKES THE ENTIRE LINE BEFORE "\N".

IF THE AMOUNT OF DATA IS UNKNOWN, THE FOLLOWING LOOP IS USEFUL:

```
while (cin >> x) {
    // code
}
```

IN SOME PROBLEMS, FILES ARE USED FOR INPUT AND OUTPUT. AN EASY SOLUTION FOR THIS IS TO WRITE THE CODE AS USUAL USING STANDARD STREAMS, BUT ADD THE FOLLOWING CODE AT THE BEGINNING:

```
freopen("input.txt", "r", stdin);
freopen("output.txt", "w", stdout);
```

THE PROGRAM READS INPUT FROM "INPUT.TXT"

THE PROGRAM WRITES THE OUTPUT TO "OUTPUT.TXT"

WORKING WITH NUMBERS:

INTEGER:

THE MOST USED INTER IS INT, WHICH IS A 32-BIT TYPE WITH A VALUE RANGE OF $-2^{31} \dots 2^{31}-1$ OR ABOUT $-2 \cdot 10^9 \dots 2 \cdot 10^9$. IF INT IS NOT ENOUGH, THE 64-BIT TYPE LONG LONG CAN BE USED. ($-2^{63} \dots 2^{63}-1$ OR ABOUT $-2 \cdot 10^{18} \dots 2 \cdot 10^{18}$).

```
long long x = 123456789123456789LL;
```

THE SUFFIX LL MEANS THAT THE TYPE OF THE NUMBER IS LONG LONG.

NOTE: A COMMON MISTAKE WHEN USING LONG LONG IS THAT THE TYPE INT IS STILL USED SOMEWHERE IN THE CODE:

```
int a = 123456789;
long long b = a*a;
cout << b << "\n"; // -1757895751
```

EVEN THOUGH THE VARIABLE "b" IS LONG LONG, BOTH NUMBER IN "a*a" ARE INT AND THE RESULT WILL BE INT (CAUSING OVERFLOW).

SOLUTION: CHANGING THE TYPE INT TO LONG LONG OR CASTING THE EXPRESSION TO (LONG LONG) a*a.

USUALLY IN CONTEST LONG LONG IS ENOUGH, STILL IS GOOD TO KNOW THA G++ COMPILER PROVIDES 128-BIT TYPE __INT128_T ($-2^{127} \dots 2^{127}-1$ OR ABOUT $-2 \cdot 10^{38} \dots 2 \cdot 10^{38}$).

MODULAR ARITHMETIC:

WE DENOTE BY "x mod m" THE REMAINDER WHEN x IS DIVIDED BY m.

EX: $17 \bmod 5 = 2 \Rightarrow 17 = (5 \cdot 3) + 2$

AN IMPORTANT PROPERTY OF THE REMAINDER IS THAT IN ADDITION, SUBCTRACTION AND MULTIPLICATION, THE REMAINDER CAN BE TAKEN BEFORE THE OPERATION:

```
(a+b)mod m = (a mod m + b mod m) mod m
(a-b) mod m = (a mod m - b mod m) mod m
(a*b) mod m = (a mod m * b mod m) mod m
```

THUS, WE CAN ASSUME THAT NUMBERS WILL NEVER BECOME TOO LARGE.

FOR EXAMPLE, THE FOLLOWIN CODE CALCULATES n!, THE FACTORIAL OF n, MODULO m:

```
long long x = 1;
for (int i = 2; i <= n; i++) {
        x = (x*i) % m;
}
cout << x%m << "\n";
```

→ FACTORIAL OF n mod m

NOTE: WE WANT THE REMAINDER TO BE BETWEN 0... m-1. HOWEVER, IN C++, THE REMAINDER OF A NEGATIVE NUMBER IS EITHER 0 OR NEGATIVE. AN EASY WAY IS TO CALCULATE THE REMINDER AS USUAL AND THEN ADD m IF THE RESULT IS NEGATIVE.

```
x = x % m;
if (x < 0) x += m;
```

FLOATING POINT NUMBERS:

THE USUAL FLOATING TYPES IN COMPETITIVE PROGRAMMING ARE THE 64-BIT DOUBLE AND 80-BIT LONG DOUBLE, BUT IN MOST CASES DOUBLE IS ENOUGH.

SOME NUMBER CANNOT BE REPRESENTED ACCURATELY AS FLOATING POINT NUMBERS, AND THERE WILL BE ROUNDING ERRORS.

EX:

```
double x = 0.3*3+0.1;
printf("%.9f\n", x); // 0.9999999999999988898
```

→ THIS SHOULD BE 1.

IT IS RISKY TO COMPARE FLOATING POINT NUMBERS USING "==" OPERATOR, A BETTER WAY IS TO COMPARE THEM IS TO ASSUME THAT TWO NUMBERS ARE EQUAL IF THE DIFFERENCE BETWEEN THEM IL LESS THAN ε, WHERE ε IS A SMALL NUMBER: $(\varepsilon = 10^{-9})$:

```
if (abs(a-b) < 1e-9) {
        // a and b are equal
}
```

SHORTENING CODE:

SHORT CODE IS IDEAL IN COMP. PROG., BECAUSE PROGRAMS SHOUL BE WRITTEN AS FAST AS POSSIBLE. PROGRAMMERS OFTEN DEFINES SHORTER NAMES FOR DATATYPES OR OTHER PARTS OF CODE.

TYPE NAMES:

```
typedef long long ll;
```

→ SHORTEN THE LONG LONG TO "LL".

AFTER THIS THE CODE:

```
long long a = 1234567;
long long b = 1234565;
cout << a*b << "\n";
```

CAN BE SHORTENED TO:

```
ll a = 1234567;
ll b = 1234565;
cout << a*b << "\n";
```

TYPEDEF CAN ALSO BE USED FOR MORE COMPLEX DATA. FOR EXAMPLE:

```
typedef vector<int> vi;
typedef pair<int,int> pi;
```

→ VECTOR OF INTEGERS

→ PAIR OF INTEGERS

MACROS:

ANOTHER WAY TO SHORTEN CODE IS TO DEFINE MACROS, WHICH MEANS THAT A CERTAIN STRING IN THE CODE WILL BE CHANGED BEFORE COMPILATION. (WITH #DEFINE KEYWORD)

EX:

```
#define F first
#define S second
#define PB push_back
#define MP make_pair
```

AFTER THIS, THE CODE:

```
v.push_back(make_pair(y1, x1));
v.push_back(make_pair(y2, x2));
int d = v[i].first + v[i].second;
```

CAN BE SHORTENED TO:

```
v.PB(MP(y1,x1));
v.PB(MP(y2,x2));
int d = v[i].F + v[i].S;
```

A MACRO CAN ALSO HAVE PARAMETERS TO SHORTER LOOPS OR OTHER STRUCTURES:

```
#define REP(i, a, b) for (int i = a; i <= b; i++)
```

```
for (int i = 1; i <= n; i++) {
        search(i);
}
```

$\Longrightarrow$

```
REP(i, 1, n) {
        search(i);
}
```

NOTE: SOMETIMES MACRO CAN CAUSE BUGS THAT MAY BE DIFFICULT TO DETECT:

```
#define SQ(a) a*a
```

$\Longrightarrow$

```
cout << SQ(3+3) << "\n";
```

$\hookrightarrow$ `cout << 3+3*3+3 << "\n";`

$\hookrightarrow$ $9+3+3=15$

```
#define SQ(a) (a)*(a)
```

$\Longrightarrow$

```
cout << SQ(3+3) << "\n";
```

$\hookrightarrow$ `cout << (3+3)*(3+3) << "\n";`

$\hookrightarrow$ $6*6=36$

MATHEMATICS:

SUM FORMULAS:

$$\boxed{\sum_{x=1}^{n} x^k = 1^k + 2^k + 3^k + \ldots + n^k}$$

WHERE K IS A POSITIVE INTEGER, HAS A CLOSED-FORM FORMULA THAT IS A POLYNOMIAL OF DEGREE K+1 :

$$\boxed{\sum_{x=1}^{n} x = 1+2+3+\ldots+n = \frac{n(n+1)}{2}}$$

AND

$$\boxed{\sum_{x=1}^{n} x^2 = 1^2+2^2+3^2+\ldots+n^2 = \frac{n(n+1)(2n+1)}{2}}$$

AN ARITHMETIC PROGRESSION IS A SEQUENCE OF NUMBER WHERE THE DIFFERENCE BETWEEN ANY 2 CONSECUTIVE NUMBERS IS COSTANT.

EX: $3, 7, 11, 15$

$\underbrace{3,7}_{4}$, $\underbrace{7,11}_{4}$, $\underbrace{11,15}_{4}$